LITHAVEN CAMPUS STUDY VAULT • TOPPER-VERIFIED REVISION CAPSULE

DSA & LeetCode 150 Patterns Master Revision Sheet

Subject: Data Structures & Algorithms | Author: Aanya Sharma (IIT Delhi) | 100% Free Campus Access

1. CORE ARRAY & STRING PATTERNS

• **Two Pointers (Opposite Direction):** Used on sorted arrays for target pairs. Left starts at 0, right at N-1. When sum < target: left++; when sum > target: right--. Time: O(N), Space: O(1).
• **Sliding Window (Dynamic):** Maintains a window [L, R] satisfying a constraint (e.g. longest substring without repeating chars). Expand R until invalid; shrink L until valid again. Maximize or minimize window length.
• **Prefix Sum & Frequency Maps:** Useful for subarray sum equals K. Maintain hashmap of cumulative sums: if (currentSum - K) is in map, increment answer count. Reduces O(N^2) to O(N).

2. LINKED LIST & FAST/SLOW POINTERS (FLOYD CYCLE)

• **Cycle Detection:** Slow advances by 1 step, Fast advances by 2 steps. If slow == fast, a cycle exists. To find cycle start: reset slow to head, advance both by 1 step until they meet.
• **In-Place Reversal:** Keep prev = null, curr = head. While curr: next = curr.next; curr.next = prev; prev = curr; curr = next. Return prev.

3. BINARY SEARCH VARIANTS

• **Upper / Lower Bound:** Never use mid = (low + high)/2 to prevent integer overflow; use low + (high - low) / 2. Shrink search space to find boundary condition or first True.
• **Search in Rotated Sorted Array:** Identify the strictly sorted half (either nums[low] <= nums[mid] or nums[mid] <= nums[high]) and check if target lies within that half.

4. DYNAMIC PROGRAMMING FORMULA CHEAT SHEET

• **0/1 Knapsack:** dp[i][w] = max(dp[i-1][w], val[i-1] + dp[i-1][w - wt[i-1]]). Space can be optimized to 1D array iterating weight backwards from W down to wt[i].
• **Longest Common Subsequence (LCS):** If s1[i-1] == s2[j-1]: dp[i][j] = 1 + dp[i-1][j-1]. Else: dp[i][j] = max(dp[i-1][j], dp[i][j-1]). Base case: dp[0][*] = dp[*][0] = 0.
• **Longest Increasing Subsequence (LIS):** O(N^2) using DP: dp[i] = 1 + max(dp[j]) for j < i and nums[j] < nums[i]. O(N log N) using Patience Sorting + Binary Search on active tails.

5. GRAPH ALGORITHMS & TIME COMPLEXITIES

• **Breadth-First Search (BFS):** Level-order traversal using Queue. Finds shortest unweighted path in O(V + E) time.
• **Dijkstra Algorithm:** Single-source shortest path for non-negative weights using Min-Heap / PriorityQueue. Time Complexity: O((V + E) log V).
• **Topological Sort (Kahn Algorithm):** Compute in-degrees for all vertices. Push 0-in-degree nodes to queue. If processed count != V, cycle exists. Essential for Course Schedule dependency problems.